Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

DISTRIBUTED MULTI-MODEL ANALYTICS FOR E-COMMERCE DATA

BIG DATA ANALYTICS FINAL PROJECT

By:
BUYANGE B. Aline
StudID: 101367
https://github.com/Buyange/ecommerce-bigdata-analytics

November, 10th, 2026

1. Introduction and System Architecture

In this project, I analysed data from an online shop. The goal was to use three tools for what each one does best: MongoDB (a document database), HBase (a wide-column database) and Apache Spark (distributed processing). I wanted to answer business questions such as: which products sell best, how often customers buy, and where visitors leave before buying.

The data comes from the provided Python generator. I generated 10,000 users, 5,000 products in 25 categories, 50,000 transactions (109,863 purchased items) and 200,000 browsing sessions (about 2 million page views). The data covers 4 August to 2 November 2026.

1.1 Architecture

Everything runs on my laptop in three Docker containers: MongoDB 7, HBase and a PySpark container (Spark 4.2.1). Figure 1 shows how the parts fit together.

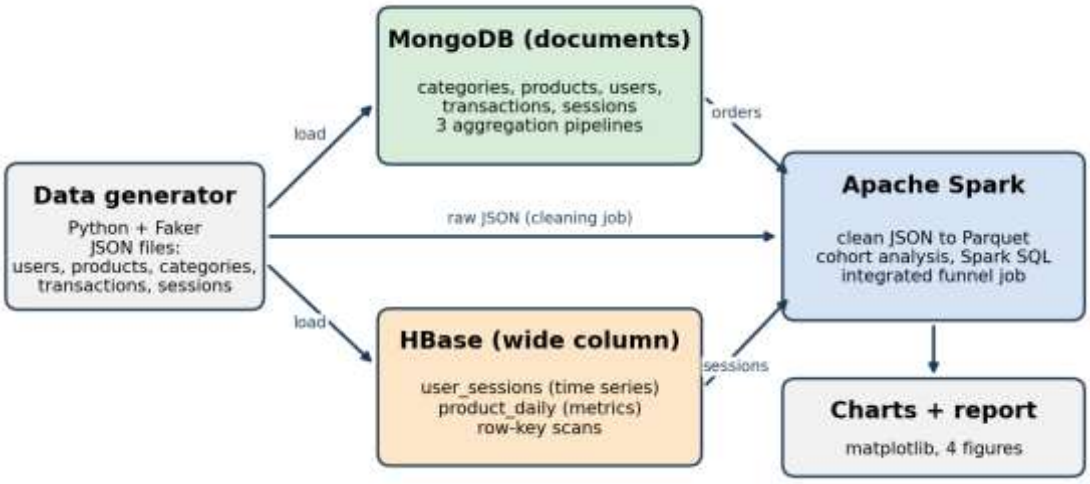


Figure 1. System architecture

- **MongoDB** stores products, users, orders and sessions as documents. I use it for sales analytics with aggregation pipelines.
- **HBase** stores the browsing sessions and daily product metrics. I use it to read the activity of one user very quickly.
- **Spark** reads the raw JSON files, cleans them and saves them as Parquet. It also runs the cohort analysis and the Spark SQL queries. In the integrated job it reads sessions from HBase and orders from MongoDB and joins them.

Spark runs in local mode (2 threads), so this is not a real cluster, but the code would be the same on one. MongoDB and HBase hold the first 100,000 sessions (the first session file); the Spark cleaning job uses all 200,000.

2. Part 1: Data Modelling

2.1 MongoDB design

In MongoDB I followed one rule: **embed** data that is always read together and stays small, and keep a **reference** (an id) for data that is shared. Table 1 shows the five collections.

Table 1. MongoDB collections

Collection	A document contains	Design choice and reason
categories (25)	A category with its subcategories (name, profit margin)	Subcategories embedded: they are only read together with their category.
products (5,000)	Product details, category id and name, price history	Price history embedded (small, belongs to one product). Category name copied in, so no join is needed to show it.
transactions (50,000)	An order with its items; each item also has category_id	Items embedded because an order is read as a whole. The category on each item lets me compute revenue by category without a join.
users (10,000)	Profile plus a purchase_summary (orders, total spent, first and last purchase)	The summary is calculated once at load time, so user segmentation reads only the users collection.
sessions (100,000)	One whole visit with page_views and cart_contents inside	The complete user journey is one document: one read gives everything.

While loading, all date strings were converted to real dates so MongoDB can group by week. Only orders with status completed, shipped or delivered count as revenue (87.5% of orders); processing orders are not counted. Product and category revenue uses the item subtotals (before the order discount), while weekly totals, segments and the cohort analysis use the order total (after discount).

Trade-off. Copying the category name and id into other documents makes reads fast but uses more space, and if a category were renamed the copies would need updating. For an analytics system that mostly reads, I think this is a good trade. **Indexes:** I created indexes for the queries I run: transactions by (user_id, timestamp), by timestamp and by items.product_id; products by category; sessions by (user_id, start_time) and by viewed_products.

2.2 MongoDB aggregation pipelines

I implemented three pipelines (mongo/aggregations.py):

- **Top-selling products:** $match (revenue orders), $unwind (one row per item), $group (units and revenue per product), $sort, $limit and $lookup (to get the product name).
- **Weekly revenue by category:** $unwind, then $group by category and $dateTrunc (week). It returned 350 rows (25 categories × 14 weeks).
- **User segments:** $bucket on the number of orders in each user's purchase_summary: no purchase, one-time, occasional (2-3), regular (4-6) and loyal (7+).

Table 2. Top 5 products by units sold

Product	Units sold	Revenue	Revenue per unit
prod_01751	87	28,811.79	331
prod_01032	77	20,631.38	268
prod_04285	76	1,560.28	21
prod_01889	74	34,334.52	464
prod_04535	74	26,691.06	361

The ranking depends on the measure. prod_04285 is third by units but earned only 1,560 (about 21 per unit), while prod_01889 sold 74 units and earned 34,335. So the best seller by volume is not always the best earner.

2.3 HBase design

HBase keeps rows sorted by row key and is very fast when I ask for one key or a range of keys. So the whole design starts from the question I want to answer, and the row key is built from it. I made two tables (Table 3).

Table 3. HBase tables

Table	Row key	Column families	Purpose
user_sessions	user_id#reverse_timestamp#session_id	meta (session info, cart), device, pv (one column per page view: pv:0001, pv:0002 ...)	Read the recent activity of one user
product_daily	product_id#yyyymmdd	m (views, cart_adds, purchases)	Product performance per day

Queries (hbase/query_hbase.py and the HBase shell): (1) the latest 5 sessions of a user, using a prefix scan on user_id; (2) the sessions of a user in a date range, using a scan between two row keys (user_007129 had 3 sessions in September 2026); (3) a row count, which confirmed that all 100,000 sessions were loaded; (4) the metrics of a product for one day (get) and for one month (range scan). The tables are filled by hbase/load_hbase.py and hbase/load_product_daily.py.

Table 4. Why some data is in MongoDB and some in HBase

	MongoDB	HBase
Strong at	Flexible queries and aggregations over nested documents	Very fast reads by row key and key ranges; huge sparse tables
Data I put there	Products, users, orders (and a session subset)	Session event time series, daily product metrics
Queries I ran	Top products, weekly revenue, user segments	Latest sessions of a user, a user's date range, product metrics per day
Weak point	Copied data must be kept up to date; large scans need indexes	No joins or ad-hoc queries; the row key must be designed first

Orders are in MongoDB because they are read as whole documents and summed in many ways. Sessions are in HBase because they are time series that I read per user. I also loaded the sessions into MongoDB to show the "whole journey in one document" strength.

3. Part2: Data Processing with Apache Spark

3.1 Reading the data

The JSON files are one big array, so I read them with multiLine = true. At first I let Spark guess the schema, and this gave a problem: cart_contents is a dictionary whose keys are product ids (different in every session), so Spark created one column per product, thousands of columns. I fixed this by writing the schema by hand (spark/schemas.py) and reading cart_contents as a

map from product id to {quantity, price}. This was also faster: 100,000 sessions were read in 5 seconds.

3.2 Data quality check

Before cleaning, I measured the data (spark/03_quality_check.py). I found no errors: all ids are unique, there are no nulls in key fields, no negative totals, every order total equals subtotal minus discount, every item price × quantity equals its subtotal, no session ends before it starts, and every referenced user and product exists. I did find these facts:

- 24,906 of the 50,000 transactions (49.8%) have no session_id.
- Order status: completed 62.5%, shipped 12.6%, processing 12.5%, delivered 12.4%.
- Sessions (first file): 44.7% abandoned, 35.1% browsed, 20.2% converted.
- Referrer (about 20% each) and device (about 33% each) are almost perfectly uniform, so differences between them would only be noise.

3.3 Cleaning job

spark/04_clean.py checks validity rules (it counts the rows that break each rule in a single pass and prints a report), removes duplicates and invalid rows, and then normalises the data. Text timestamps become real timestamps; I added is_revenue, txn_date and registration_month; and the nested lists were flattened into tables with one row per item: transaction_items (109,863 rows), session_page_views (1,998,578 rows) and session_cart_items (208,004 rows). Because the data was already clean, every rule removed 0 rows. The rules are there to protect the pipeline if the data changes.

Why Parquet? The results are saved as Parquet files. JSON has to be parsed again every time, but Parquet is compressed, column-based and keeps the schema, so later jobs read only the columns they need and start much faster. The cleaned sessions are written once to a staging folder and the three session tables are built from that copy, so the large JSON files are parsed only twice.

3.4 Cohort analysis

For the batch analysis I chose a cohort analysis (spark/05_cohort.py). Users are grouped by registration month (February to August 2026; cohort sizes from 195 to 1,739 users). For each cohort and purchase month I calculated buyers, orders, revenue and revenue per user (revenue divided by the cohort size, so cohorts of different size can be compared). Revenue per user per day is about 50 for every cohort in every month (range 47.3 to 52.3, Figure 3), so registration month does not change spending. August has only 28 days of data and November only 2, so I compared revenue per day and ignored November.

3.5 Spark SQL

I registered the Parquet tables as SQL views and wrote three queries (spark/06_sql.py). In the code, each view is labelled with the system it would normally come from (MongoDB or HBase), to show how data from both could be joined in Spark.

- **Q1:** top 5 categories per month and their share of revenue (window functions RANK and SUM OVER). In September and October every category has only 4.3% to 5.0% of revenue (an equal share would be 4%); cat_004 is first in both months.
- **Q2:** top 3 products inside each category (ROW_NUMBER over partitions), shown below.
- **Q3:** engagement vs spending. It joins session data (HBase-style) with orders (MongoDB-style) and splits users into four equal groups by number of sessions (NTILE).

Table 5. Result of Q3: users by number of sessions

Group	Users	Sessions per user	Avg orders	Avg revenue
1 (lowest)	2,500	6 - 17	3.70	4,006
2	2,500	17 - 20	4.18	4,364
3	2,500	20 - 23	4.53	4,631
4 (highest)	2,500	23 - 40	5.10	5,135

Users with more sessions place more orders (3.7 orders in the lowest group, 5.1 in the highest). I think this link is built into the generator, because orders are created from converted sessions, so I describe it as an observation and not as a discovery.

Optimisation choices: an explicit schema instead of schema guessing; Parquet instead of repeated JSON parsing; all validity rules counted in one pass; sessions staged once; and the joined DataFrame cached in the funnel job. Spark also warned that NTILE has no PARTITION BY, so all rows go to one partition. This is fine for 10,000 users but would be a bottleneck at large scale.

4. Part 3: Integrated Analytics: Funnel Conversion

Business question: where do visitors drop out between looking at a product and buying it?

4.1 Data sources and workflow

Table 6. Funnel stages and where each one comes from

Stage	Source	Rule
1. Viewed a product	HBase	The session has a product_detail page view
2. Added to cart	HBase	Stage 1 and the cart is not empty
3. Reached checkout	HBase	Stage 2 and the session has a checkout page view
4. Purchased	MongoDB	Stage 3 and an order with this session_id exists

The job spark/08_funnel.py runs inside the Spark container, which reaches the other containers by service name:

1. Python reads the 100,000 sessions from HBase through the Thrift server (happybase) and keeps the page types, cart size, device and referrer.
2. Python reads the 25,094 session-linked orders from MongoDB (pymongo).
3. Spark makes two DataFrames and left-joins the sessions with the orders on session_id.
4. Spark calculates the four cumulative stages with array functions and counts them overall and by device.
5. Consistency checks compare the two databases.

4.2 Results

Table 7. Conversion funnel (100,000 sessions)

Stage	Sessions	% of all	% of previous stage
All sessions	100,000	100.0	-
Viewed a product	98,716	98.7	98.7
Added to cart	64,939	64.9	65.8

Stage	Sessions	% of all	% of previous stage
Reached checkout	29,023	29.0	44.7
Purchased (order in MongoDB)	20,239	20.2	69.7

The biggest loss is from cart to checkout: 55% of the sessions with a cart never reach checkout, and another 30% leave at checkout. Only 31% of the sessions with a non-empty cart end in a purchase (cart abandonment is about 69%). The purchase rate is the same across devices: desktop 20.3%, tablet 20.2%, and mobile 20.2%.

Checks between the databases: all 20,239 sessions that HBase marks as converted have an order in MongoDB, and no abandoned or browsed session has one. All 25,094 session-linked orders belong to different sessions, and 4,855 of them belong to the second session file, which is not in HBase (25,094 - 4,855 = 20,239), so everything matches.

4.3 Why these technologies, and performance

HBase is the natural place for session events, MongoDB for orders, and Spark for the join and the counting. The slow part is reading 100,000 rows through one Python process over Thrift (Spark also warned that its tasks were large because the data was built from Python lists). At a larger scale I would read HBase in parallel (one scan per region) or export it to Parquet first.

5. Part 4: Key Findings and Visualizations

Methodology. Revenue counts only completed, shipped, and delivered orders. Weekly values use weeks that start on Sunday and leave out the first week (only 4-8 August) and the last week (only 1-2 November), which are partial. Segments use the number of orders per user. The charts were made with matplotlib from MongoDB results and from the CSV files that the Spark jobs saved in the results folder.

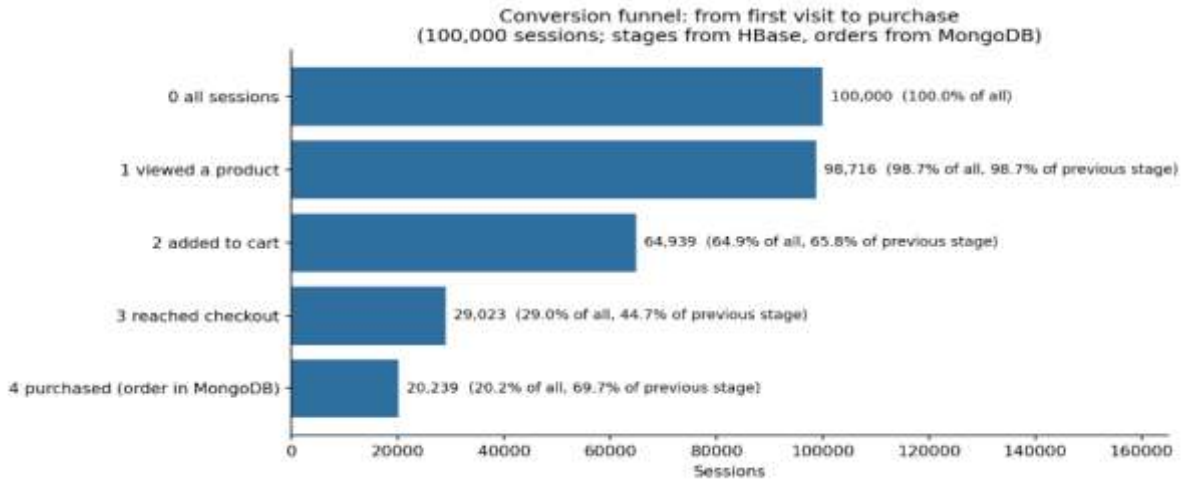


Figure 2. Conversion funnel (HBase sessions + MongoDB orders)

Finding 1: carts are the main leak. Only 44.7% of the sessions with a cart reach checkout. Reminders for abandoned carts and showing the total cost earlier would be the first things to try. Device and referrer make no difference in this data.

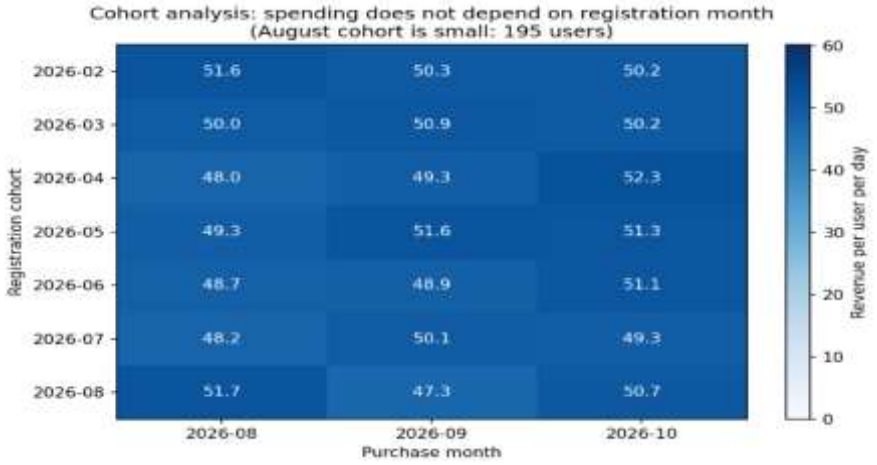


Figure 3. Revenue per user per day by registration cohort

Finding 2: no cohort effect. Every cell is close to 50, so older and newer users spend the same. In real data I would expect differences, so this is a property of the synthetic data.

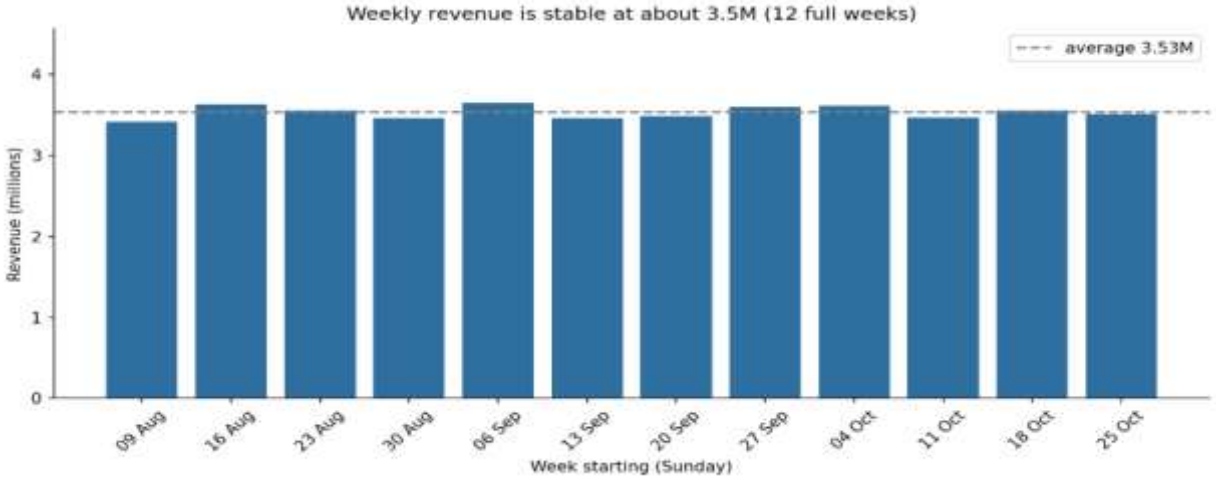


Figure 4. Weekly revenue (12 full weeks)

Finding 3: stable revenue. Weekly revenue is between 3.42M and 3.65M, with an average of 3.53M, and there is no trend. There is no seasonality in 90 days of this data.

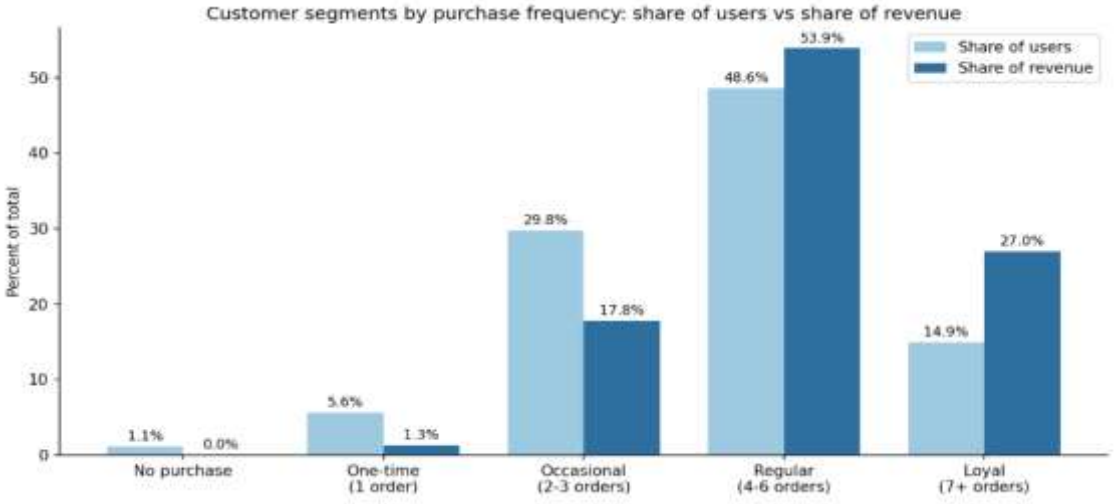


Figure 5. Customer segments: share of users vs share of revenue

Finding 4: loyal customers are worth the most. Loyal buyers (7+ orders) are 14.9% of users but 27.0% of revenue, and each spends on average 8,211, about eight times that of a one-time buyer (1,038). Regular buyers are the largest group (48.6% of users, 53.9% of revenue). Occasional buyers (29.8% of users, 17.8% of revenue) are the best group to move up.

6. Scalability and Technology Selection

6.1 Scalability

- **MongoDB:** the indexes already match the queries. For much more data, I would shard the collections, for example, by hashed user_id. Embedding page views inside a session is fine because a session is small.
- **HBase:** tables split into regions automatically. The row keys start with user_id or product_id, which are spread out, so the writes do not all go to one place (no hot spot). The reverse timestamp keeps the newest rows of a user together.
- **Spark:** the same code can run on a cluster. Parquet is column-based, so jobs read less data. The large JSON files with multiLine cannot be split, so each file is read by one task; converting them to Parquet once (what the cleaning job does) removes this problem. The NTILE window without partitions and the single-process HBase read are the two places that would need a redesign at a large scale.

7. Insights and recommendations

7.1 Limitations

- **The data is synthetic and very uniform.** Results such as "no cohort effect" or "no device effect" say more about the generator than about real customers.
- Users have no age, so I could not segment by age; products have no subcategory, so I could not analyse profit margin per product.
- Revenue is defined in two ways (item subtotal before discount for products and categories, order total after discount for everything else). Both are explained in Section 2.1.
- About half of the transactions have no session. The generator reached its 50,000 transaction target after about 130,000 sessions, so some later sessions may be marked converted without an order. MongoDB and HBase hold only the first 100,000 sessions.
- Everything runs on one laptop (Spark in local mode), so I could not measure real distributed performance.
- The data covers only 90 days, and I did not run statistical tests, so the findings are descriptive.
- The brief mentions simulated stream processing, but I implemented only batch jobs.

7.2 Future work

- Add the "users who bought X also bought Y" analysis, and a simulated stream with Spark Structured Streaming that reads session files as they arrive.
- Use the MongoDB and HBase connectors for Spark, and run on a real cluster with the full 2 million sessions.

7.3 Business Insights and Recommendations

The analysis identified the following opportunities to improve business performance:

- **Reduce cart abandonment:** Only 44.7% of sessions with a cart proceed to checkout. Introduce abandoned-cart reminders, display full purchase costs, and simplify checkout.
- **Improve customer retention:** Loyal customers represent 14.9% of users but generate 27.0% of revenue. Introduce loyalty rewards and targeted offers to encourage repeat purchases.
- **Convert occasional buyers:** Occasional buyers represent 29.8% of users but contribute 17.8% of revenue. Use follow-up promotions to encourage more frequent purchases.
- **Estimated business impact:** A one-percentage-point improvement in cart-to-checkout conversion could generate approximately 450 additional purchases. Similarly, converting 10% of occasional buyers (approximately 300 users) into regular buyers could increase estimated revenue by approximately 0.7M. These are illustrative scenarios, not forecasts.
- **Architecture contribution:** MongoDB supports business data storage and reporting, HBase enables efficient retrieval of customer session data, and Apache Spark performs large-scale data processing and analytics.

7.4 Conclusion

The project demonstrates how MongoDB, HBase, and Apache Spark work together to support e-commerce analytics. The findings highlight opportunities to improve checkout conversion, customer retention, and product performance. Since the analysis uses synthetic data, the recommendations should be validated using real business data before implementation.